

1 | Introduction

This report evaluates two methods for forecasting UK carbon intensity (CI) at monthly and hourly resolutions, using data from 2009-2024 for training and from 2025 for validation, obtained from NESOⁱ.

Forecast accuracy is measured using mean absolute error (MAE) and root mean squared error (RMSE), and results are compared to a persistence baseline based on 2024 CI values. Carbon emissions associated with code execution are estimated using real-time CI in London from the UK Carbon Intensity APIⁱⁱ.

2 | Experimental Setup

The code was implemented in Python and executed in the Visual Studio Code (VSC) terminal on a Windows 11 laptop with an Intel i7-13700H CPU and 16 GB RAM. Third-party libraries used include *NumPy*, *Pandas*, *Matplotlib*, *Statsmodels*, *SciPy*, *scikit-learn*, and *LibreHardwareMonitor*.

Runtime measurements are reported as the mean of five trials and conducted with non-essential applications closed and the device connected to mains power. Power consumption was sampled in parallel using LibreHardwareMonitor at its maximum supported polling rate (250ms).

No generative AI was used in the development of code or the writing of this report.

3 | Prediction Methods

Figures 1 and 2 summarise the training data structure. The monthly CI exhibits two distinct regimes, which motivates piecewise modelling, while we observe a positive relationship between seasonal amplitude and CI, consistent with multiplicative seasonal effects. The mean weekly hourly profile demonstrates a strengthening bimodal diurnal cycle and growing weekday-weekend divide, encouraging use of recent data.

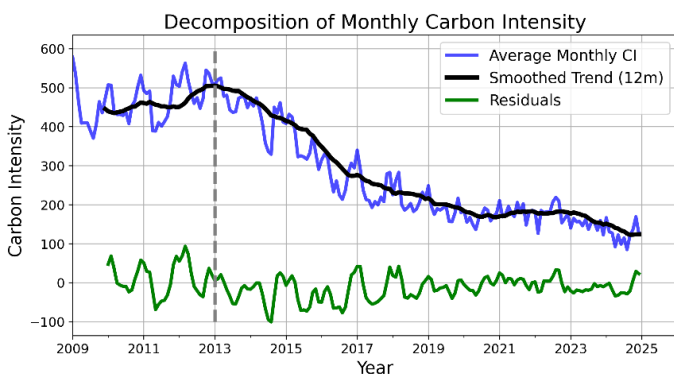


Figure 1: Trend-residual decomposition of monthly training data using a 12-month smoothing window

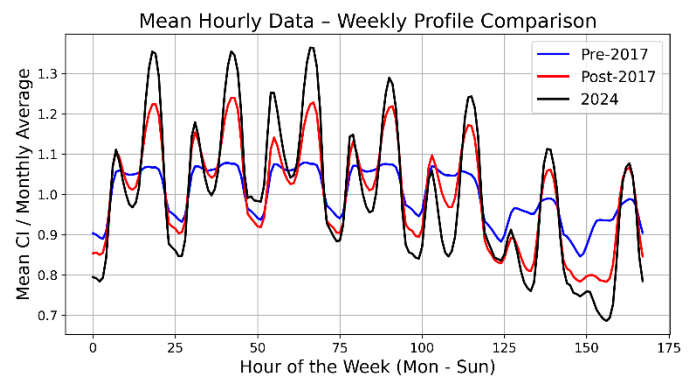


Figure 2: Mean weekly profile for hourly CI relative to monthly average, across different training periods

3.1 | Recomposition Model: Multiplicative Trend Decomposition

In our model $CI(t) = T_{trend}(t)T_{month}(m)T_{hour}(h)$, the long-term trend was approximated as a linear fit over data from 2023-2024 and extrapolated to 2025. Monthly seasonality used mean monthly deviation from the annual average across all years, reflecting the stable yearly cycle in *Figure 1*. Finally, the systematic changes in the weekly hourly profile prompted use of only the 2024 pattern for forecasting.

3.2 | Regression Model

A linear regression served as a baseline, while analysis focused on a piecewise model and extended nonlinear model. Monthly models were evaluated using rolling-origin validation (RV) over 2016-2024, with recent years weighted more heavily. The piecewise model comprised a linear then log-linear regime, and although visually appropriate, $\{CI(t), \ln(CI(t))\} = \beta_1 + \beta_2 t + \beta_3 f_{2\pi/12}(m)$, where $f_\omega(x) = \begin{pmatrix} \sin(\omega x) \\ \cos(\omega x) \end{pmatrix}$, constrained seasonal amplitude to evolve with the long-term trend and limited independence between them (*Figure 3*).

The nonlinear model $CI(t) = A(1 + t/\tau)^{-p} + \mathbf{X}(m, h, q)\boldsymbol{\beta}$, with $q(t) = (1 + t/\tau_s)^{-1}$, captured long-term decay parametrically while applying Ridge regression to the residuals to reduce multicollinearity. The feature set $\mathbf{X}(m, h, q)$ included monthly (m) and hourly (h) harmonics, decay modulation (q), and a weekday-weekend structure (*Appendix A, B, C, D*). A Cartesian grid search using RV-MAE optimised the breakpoint

t_b , seasonal modulation timescale τ_s , and Ridge

parameter α for the monthly model; the hourly model

reoptimised α only. As shown in *Table 1*, this decoupling

of trend and residual evolution reduced monthly RV-MAE

by 21% and 49% relative to the piecewise and linear

models, respectively, at a $35\times$ and $821\times$ runtime cost.

State	Regression		Recomposition		Overall
	Monthly	Hourly	Monthly	Hourly	
Initial	205.23	4.00	0.04	0.33	211.09
A	11.10	3.93	0.04	0.32	16.86
B	1.75	2.14	0.02	0.31	5.68
Final / C	1.74	2.15	0.02	0.06	5.45

Table 2: Impact of successive system-level optimisations on runtime (seconds) of regression and recomposition methods

Metric	Linear	Piecewise	Nonlinear
RV-MAE	47.1	30.8	24.2
Training MAE	44.0	30.9	23.8
Runtime	0.25	5.81	205.23

Table 1: Performance of monthly regression models, including MAE (gCO2/kWh) and runtime (seconds)

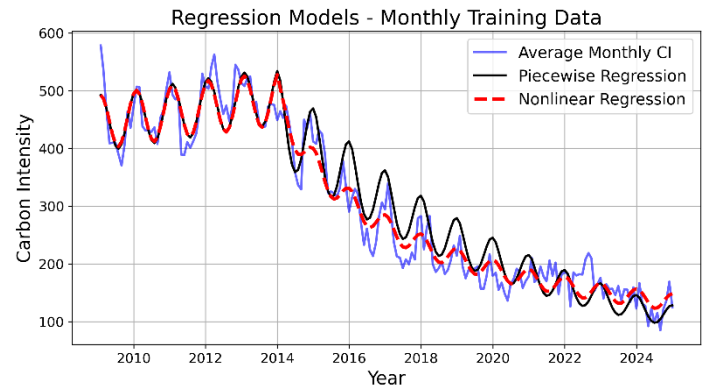


Figure 3: Comparison of piecewise and nonlinear regression model fits on monthly training data

4 | System Performance Optimisation Techniques

Optimisation A: The Cartesian grid search for monthly regression hyperparameters required 768 RV-MAE evaluations. This reduced to 44 evaluations by employing per-parameter gradient descent until convergence.

Optimisation B: Caching intermediate results, including resampled pandas datasets, sinusoidal feature vectors, pre-breakpoint regression fits, and power-law variables, eliminated repeated computation overheads.

Optimisation C: Replacing Python loops in the recomposition method with vectorised column-wise multiplication in pandas delegated computation to faster, compiled routines.

Table 2 shows an overall code speedup of $38\times$, with no change in accuracy for any optimisation. If we assume an average power of 23.4 W (Figure 6) and a carbon intensity of $177 \text{ gCO}_2/\text{kWh}$ in London (4th January 2026, 17:30 GMT), reducing runtime from 211.09s to 5.45s lowers emissions by 236.59 mgCO_2 .

5 | Results and Discussion

Metric	Regression		Recomposition		Persistence	
	Monthly	Hourly	Monthly	Hourly	Monthly	Hourly
MAE	11.8	45.8	22.6	45.6	16.2	56.3
RMSE	14.0	54.7	25.6	57.2	20.3	70.7
Runtime	1.74	2.15	0.02	0.06	-	-

Table 3: Evaluation of forecast errors (gCO_2/kWh) and runtime (seconds) of our models, compared to the persistence baseline

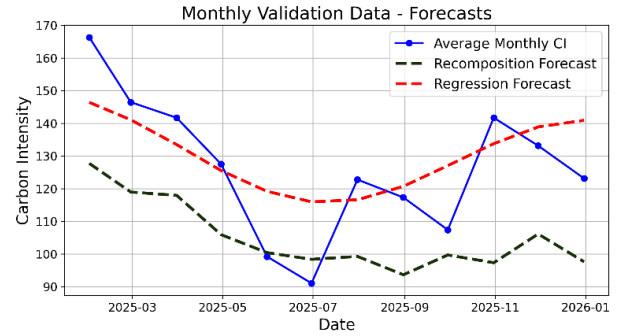


Figure 4: Monthly CI forecast of both methods

At monthly resolution, Figure 4 reveals the recomposition model struggles with negative forecast bias and underestimated seasonal amplitudes. Table 3 shows it is less accurate than even the persistence baseline. This indicates that the linear trend estimate does not generalise well and supports independent trend and residual evolution, an issue also faced by the log-linear regression. By contrast, the nonlinear regression performs best, as it benefits from a user-guided structure that forces generalisation and suppresses noise.

At the hourly level, the high short-term variability in Figure 5 reflects the dominance of exogenous variables, such as weather conditions affecting UK wind turbine output. Thus, the models capture the mean weekly structure in Figure 2, shaped by consistent energy demand, but cannot reduce variance-based errors. This saturates their predictive potential and reduces the benefit from added complexity, yielding similarly poor performance from both models and favouring the lower computational cost of recomposition.

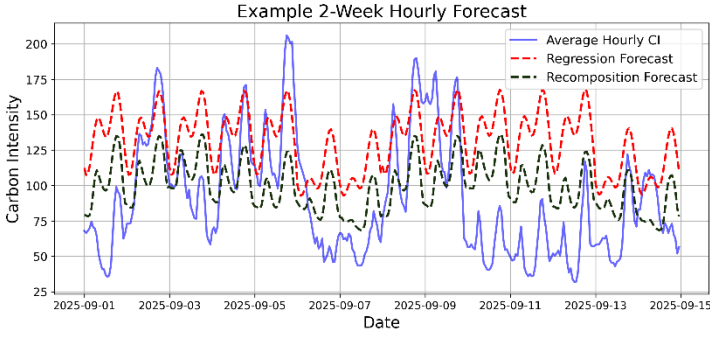


Figure 5: Sample (2-week) hourly CI forecast of models

6 | Carbon Emissions Analysis

Both execution environments, Windows PowerShell and the VSC terminal, had overall runtimes of 5.6s. However, PowerShell drew an average of 3W less power, resulting in 13% lower carbon emissions (*Appendix E*). This indicates that VSC incurs additional overheads despite equivalent execution speed.

All CI values in *Table 4* were obtained from ElectricityMapsⁱⁱⁱ for

consistency. Great Britain exhibited the greatest CI variability due to its reliance on weather-dependent renewables (e.g. wind), whereas North Sweden and New York showed consistently low and high CI levels, respectively, reflecting their predominantly renewable and gas-based energy mixes.

These results highlight the importance of coding environment, location, and timing in sustainable coding practices, especially in continuously running projects where carbon emissions will accumulate.

7 | Conclusion

In ‘one-off’ projects such as this, carbon emissions are dominated by code testing over final execution, so it is best practice to critically consider data before method selection. Here, the monthly regression achieved an $MAE < 12 \text{ gCO}_2/\text{kWh}$ through contextual design, while awareness of exogenous variables favoured the lightweight recomposition as an informative estimate for sustainability. For a higher upfront cost, regression offers greater accuracy, scales with problem size, and sustains benefits for continuous forecasting. However, the lower cost and fixed benefit of the recomposition are better suited to this ‘one-off’ problem size, underlining the need to match model complexity to achievable accuracy and workload in future projects.

Location	GMT	CI	Emissions
Great Britain	02:00	67	2.12
	10:00	93	2.94
	18:00	167	5.28
North Sweden	02:00	13	0.41
	10:00	13	0.41
	18:00	13	0.41
New York	02:00	319	10.09
	10:00	330	10.44
	18:00	324	10.25

Table 4: Carbon emissions (mg) of executed code (113.9 J) on 2nd January 2026

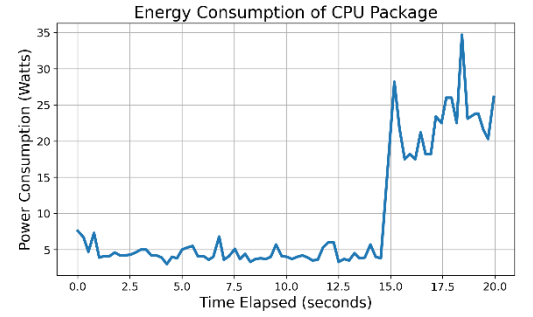


Figure 6: Live power poll of CPU Package, including a 15-second static power reading.

8 | References

- ‘Carbon Intensity API’. Accessed 6 January 2026. <https://api.carbonintensity.org.uk/>.
- ‘Historic Generation Mix and Carbon Intensity | National Energy System Operator’. 5 January 2026. <https://www.neso.energy/data-portal/historic-generation-mix>.
- ‘Interactive App | Electricity Maps’. Accessed 6 January 2026. <https://app.electricitymaps.com/map/live/hourly>.

9 | Appendix

A | Notation

In this report, t denotes time in months since the start of the dataset (or since the most recent breakpoint), while m and h index the month of the year and hour of the day respectively.

B | Rolling-origin Validation for Regression Method

The rolling-origin validation over years 2016-2024 was used to evaluate regression models. More recent years were weighted more heavily, using the normalised series $\frac{1}{4.044} \exp(-\frac{2024-year}{4})$.

C | Piecewise Regression

Our piecewise regression had general formula and hyperparameter set:

$$\{CI(t), \ln(CI(t))\} = \beta_1 + \beta_2 t + \beta_3 f_{2\pi/12}(m), \text{ with } f_{\omega}(x) = \begin{pmatrix} \sin(\omega x) \\ \cos(\omega x) \end{pmatrix}$$

$$t_b = \text{range}(48, 61)$$

and optimised our hyperparameter to $t_b = 60$.

D | Nonlinear Regression

Our nonlinear regression had general formula:

$$CI(t) = A(1 + t/\tau)^{-p} + X(m, h, q)\beta, \text{ with } q(t) = (1 + t/\tau_s)^{-1}.$$

The monthly feature set comprised the following design matrix and hyperparameter set:

$$X(m, h, q) = [1, f_{\frac{2\pi}{12}}(m), qf_{\frac{2\pi}{12}}(m)]$$

$$t_b = \text{range}(48, 61), \tau_s = [6, 12, 18, 24, 30, 36, 48, 60], \alpha = [0.1, 0.5, 1, 2, 3, 5, 7, 10]$$

and optimised our three hyperparameters to $(t_b, \tau_s, \alpha) = (60, 36, 10)$.

The hourly feature set comprised the following design matrix and hyperparameter set:

$$\mathbf{X}(m, h, q) = [1, f_{\frac{2\pi}{12}}(m), qf_{\frac{2\pi}{12}}(m), f_{\frac{2\pi}{12}}(h), f_{\frac{2\pi}{24}}(h), 1_{wknd}, 1_{wknd}f_{\frac{2\pi}{12}}(h), 1_{wknd}f_{\frac{2\pi}{24}}(h)]$$

$$\alpha = [50, 100, 150, 200, 250, 300, 350, 400]$$

and optimised our hyperparameter to $\alpha = 300$, while t_b and τ_s retained their monthly units.

E | Tabular Data for Carbon Emissions in Different Execution Environments

Environment	Trial	Static (W)	Running (W)	Runtime (s)	Energy (J)	Emissions (mg)
<i>VSC terminal</i>	#1	6.66	23.79	5.65	134.32	6.60
	#2	7.94	22.69	5.66	128.36	6.31
	#3	7.34	22.97	5.53	126.91	6.24
	#4	7.33	23.32	5.69	132.69	6.52
	#5	7.79	24.21	5.53	133.86	6.58
	Average	7.4	23.4	5.6	131.2	6.45
<i>Windows PowerShell</i>	#1	5.63	20.39	5.71	116.47	5.73
	#2	5.87	21.11	5.47	115.45	5.68
	#3	7.67	19.8	5.76	113.99	5.60
	#4	5.22	20.14	5.50	110.79	5.45
	#5	5.72	20.69	5.46	113.01	5.56
	Average	6.0	20.4	5.6	113.9	5.60

ⁱ ‘Historic Generation Mix and Carbon Intensity | National Energy System Operator’.

ⁱⁱ ‘Carbon Intensity API’.

ⁱⁱⁱ ‘Interactive App | Electricity Maps’.